Sydney Opal Patronage — Data Analyst Portfolio Project

A complete, self-paced curriculum to build one polished, interactive data project and deploy it to a live website.

This project is designed for someone returning to data analytics after a break. It deliberately spans the *trusted* stack (SQL, Power BI) and the *current* stack (Python, Git/GitHub, a small machine-learning model, and a live Streamlit web app). By the end you'll have a public GitHub repo and a live interactive website you can put on your CV and walk an interviewer through.

Realistic time: roughly 10–12 focused hours total, done in whatever order and pace suits. Don't rush the README and the deploy step — they're where a lot of the value sits.

The golden rule: *Finish a small thing completely.* One clean, fully-understood project beats three half-built ambitious ones. Resist scope creep.

What you'll build

A web app where a visitor can: - Pick a transport mode (train, bus, ferry, light rail) and a Sydney region from dropdowns - Drag a date-range slider - Watch a patronage-over-time chart update live — including the dramatic COVID collapse and recovery from 2020 onward - See a simple forecast/prediction of patronage

...backed by a GitHub repo containing clean SQL, a Python analysis notebook, a trained model, and a README that tells the whole story.

Skills checklist (what this demonstrates to employers)

- [ ] **SQL** — loading, joining, aggregating real data
- [ ] **Python (pandas)** — cleaning, exploration, analysis
- [ ] **Data visualisation** — matplotlib/plotly + an interactive dashboard
- [ ] **A predictive model** — scikit-learn, explained honestly
- [ ] **Git/GitHub** — version control with a sensible commit history
- [ ] **Deployment** — a live, shareable web app
- [ ] **Communication** — a README that frames the question, method, findings, and limitations
- [ ] **Power BI** (optional parallel track) — for the CV keyword + a screenshot

---

Prerequisites & setup (Module 0)

0.1 Install the tools (free)

Tool	What for	Where
Python 3.11+	Everything	python.org or via Anaconda
VS Code	Editor	code.visualstudio.com
Git	Version control	git-scm.com
A GitHub account	Hosting the repo	github.com
DB Browser for SQLite	Viewing the database (optional, friendly)	sqlitebrowser.org
Power BI Desktop (optional)	The CV-keyword dashboard	Microsoft Store (Windows only)

0.2 Refresher resources (use only if rusty — don't binge)

- **SQL:** SQLBolt (free, interactive) — 1–2 hours to shake off rust
- **pandas:** the official "10 minutes to pandas" guide
- **Git:** the "Git and GitHub for Beginners" basics — just `add`, `commit`, `push`, `clone`
- **Streamlit:** the official "Get started" page — 30 minutes

A returning analyst usually doesn't need full courses — a quick skim to refresh syntax is enough. Learn by building this, not by watching.

Module 1 — Project setup & first commit (~1 hour)

Goal: an empty but professional repo skeleton, pushed to GitHub.

1.1 Create the folder structure

```
opal-patronage-analysis/
├── data/              # raw + cleaned data files
├── notebooks/         # Jupyter analysis
├── sql/               # .sql query files
├── src/               # reusable python scripts
├── app/               # the Streamlit app
├── images/            # exported charts for the README
├── .gitignore
├── requirements.txt
└── README.md
```

1.2 Set up the Python environment

```
cd opal-patronage-analysis
python -m venv venv
# Windows:
venv\Scripts\activate
# Mac/Linux:
source venv/bin/activate

pip install pandas matplotlib plotly scikit-learn streamlit jupyter requests
pip freeze > requirements.txt
```

1.3 Create a `.gitignore`

```
venv/
__pycache__/
.ipynb_checkpoints/
*.pyc
.DS_Store
```

1.4 First commit & push

```
git init
git add .
git commit -m "Initial project skeleton"
# create an empty repo on github.com first, then:
git remote add origin https://github.com/YOUR_USERNAME/opal-patronage-analysis.git
git branch -M main
git push -u origin main
```

✅ **Checkpoint:** your empty structured repo is live on GitHub. That commit history starts now and tells a story — commit often from here.

Module 2 — Get the data (~1 hour)

Goal: raw Opal patronage data sitting in your `data/` folder.

2.1 The dataset

Opal Patronage, Transport for NSW — daily patronage for train, bus, ferry and light rail since January 2020, by mode, day of week, and key Sydney/NSW regions. Creative Commons licensed (free to use with attribution).

- Landing page: https://opendata.transport.nsw.gov.au/dataset/opal-patronage
- Files are published **one per day** (e.g. `Opal_Patronage_20200112.txt`), so a full date range means hundreds of files — the download loop in 2.3 matters.

2.2 The actual file structure

Pipe-delimited (`|`) text files, **hourly** granularity, with these columns:

Field	Type	Meaning
`trip_origin_date`	date	the day of travel (one file = one day)
`mode_name`	text	Bus / Train / Ferry / Light rail
`ti_region`	text	Sydney CBD, Parramatta, Chatswood, Macquarie Park, North Sydney, Strathfield, Newcastle and surrounds, Wollongong and surrounds, Other — **plus an `All - NSW` aggregate row**
`tap_hour`	number	hour of day, 0–23
`Tap_Ons`	text ⚠	tap-on count for that hour — rounded to the nearest 100, and **suppressed as the literal string `"<50"`** for small counts
`Tap_Offs`	text ⚠	tap-off count, same rounding and suppression

> **Three traps to know before writing any code:** 1. **`"<50"` values make the count columns text, not numbers.** Any sum or plot will fail (or silently misbehave) until you convert them. In one sample day, ~26% of rows were suppressed — this is not a rare edge case. 2. **`All - NSW` is a pre-computed total row.** If you `SUM()` across all regions including it, you double-count. Either filter it out when grouping by region, or use it *alone* for state-wide totals. 3. **Counts are rounded to the nearest 100**, so small differences aren't meaningful — worth a sentence in your README's limitations.

2.3 Downloading many daily files

The "download whole dataset" button is known to error sometimes, and with one file per day you don't want to click hundreds of links anyway — write a small Python loop to fetch them (a tidy first real commit). To keep the repo light, consider a subset: 2020 (the COVID collapse) plus the most recent 12–18 months (the recovery) still tells the full story.

2.4 Note the data-quality caveat

This dataset has had periods of under-reporting (e.g. missing-batch issues where some days record implausibly few trips). **Check the dataset's own data-quality notes** on the TfNSW page for the specific dates/periods they flag, and confirm them against the data itself in Module 3. Don't take a number secondhand — verify it from the source. Handling this properly in your cleaning step, and mentioning it in the README, is exactly the kind of detail that impresses interviewers.

✅ **Checkpoint:** raw data in `data/`, columns understood, caveat noted. Commit: `"Add raw Opal patronage data"`.

Module 3 — Clean & load with SQL (~2 hours)

Goal: clean data living in a SQLite database, queried with real SQL.

3.1 Load raw files into a dataframe (Python)

In `notebooks/01_data_prep.ipynb`:

```
import pandas as pd
import glob

# The Opal files are pipe-delimited (|). Load the count columns as strings
# deliberately — they contain "<50" suppression markers, so they are NOT numeric yet.
files = glob.glob("../data/raw/**/*.txt", recursive=True)   # walks subfolders too
df = pd.concat(
    (pd.read_csv(f, sep="|", dtype={"Tap_Ons": str, "Tap_Offs": str}) for f in files),
    ignore_index=True
)

# Inspect
print(f"Loaded {len(files)} files, {df.shape[0]:,} rows")
print(df.head())
print(df.dtypes)          # Tap_Ons / Tap_Offs will show as object (string) — expected
print(df['Tap_Ons'].str.contains('<').mean())  # what share of rows is suppressed?
```

> Columns you now have: `trip_origin_date | mode_name | ti_region | tap_hour | Tap_Ons | Tap_Offs` — one row per mode × region × **hour**.

3.2 Clean

Three real problems to solve here, in order: the `"<50"` suppression, the hourly→daily aggregation, and the `All - NSW` double-count trap.

```
# --- 1. Handle the "<50" suppression -------------------------------------
# Small counts are privacy-suppressed as the literal string "<50".
# Policy decision (state it in your README): treat suppressed cells as 25 —
# the midpoint of the possible 0–49 range. Alternatives: 0 (undercounts) or
# NaN (drops them). Midpoint is a common, defensible choice; what matters
# is that you CHOSE and DOCUMENTED it, not which one you picked.
df['suppressed'] = df['Tap_Ons'].str.strip().eq('<50')

for col in ['Tap_Ons', 'Tap_Offs']:
    df[col] = (df[col].str.strip()
                      .replace('<50', '25')
                      .astype(int))

# --- 2. Parse dates and standardise text ---------------------------------
df['trip_origin_date'] = pd.to_datetime(df['trip_origin_date'])
df['mode_name'] = df['mode_name'].str.strip()          # values: Bus, Train, Ferry, Light rail
df['ti_region'] = df['ti_region'].str.strip()

# --- 3. Aggregate hourly rows to a daily table ---------------------------
# Use Tap_Ons as the "trips" measure (a tap-on ≈ a journey start).
# Keep the All - NSW aggregate SEPARATE from the per-region rows so no
# query ever double-counts.
daily = (df.groupby(['trip_origin_date', 'mode_name', 'ti_region'], as_index=False)
           .agg(trips=('Tap_Ons', 'sum'),
                tap_offs=('Tap_Offs', 'sum'),
                suppressed_hours=('suppressed', 'sum')))

daily['is_nsw_total'] = daily['ti_region'].eq('All - NSW')

# --- 4. Flag anomalous / low-quality dates -------------------------------
# Don't hardcode a guessed list — FIND bad dates two ways:
#   (a) the dataset's official data-quality notes on the TfNSW page;
#   (b) detection from the data itself, e.g. days far below the local median.
chk = daily[daily['is_nsw_total']].copy()          # state-wide series per mode
chk['med'] = (chk.groupby('mode_name')['trips']
                 .transform(lambda s: s.rolling(7, center=True, min_periods=1).median()))
chk['suspect'] = chk['trips'] < 0.4 * chk['med']   # <40% of local median

# Inspect what got flagged BEFORE excluding anything — never trust the rule blindly.
# Note: genuine events (lockdowns, strikes) will flag too; that's a judgement call
# for the README, not an automatic delete.
print(chk[chk['suspect']][['mode_name', 'trip_origin_date', 'trips', 'med']])

suspect_dates = set(chk.loc[chk['suspect'], 'trip_origin_date'])
daily['data_quality_flag'] = daily['trip_origin_date'].isin(suspect_dates).astype(int)

# --- 5. Add calendar columns for later analysis --------------------------
daily['day_of_week'] = daily['trip_origin_date'].dt.day_name()
daily['is_weekend']  = (daily['trip_origin_date'].dt.dayofweek >= 5).astype(int)
daily['year']        = daily['trip_origin_date'].dt.year
daily['month']       = daily['trip_origin_date'].dt.month
```

> **Why keep the hourly detail too?** You don't have to — but the hourly `tap_hour` column enables a nice bonus chart (morning/evening peak profiles, and how they flattened post-COVID). If repo size allows, write both tables to SQLite; if space is tight, the daily table alone carries the project.

3.3 Write to SQLite

```
import sqlite3
conn = sqlite3.connect("../data/opal.db")
daily.to_sql("patronage", conn, if_exists="replace", index=False)   # daily table (main)
df.to_sql("patronage_hourly", conn, if_exists="replace", index=False)  # optional
conn.close()
```

3.4 Now actually use SQL (this is the point)

In `sql/exploration.sql` — write and run these (in DB Browser, or from Python with `pd.read_sql`):

```
-- Total trips by mode (state-wide: use the All - NSW aggregate rows ONLY,
-- otherwise you double-count region rows + the total row)
SELECT mode_name, SUM(trips) AS total_trips
FROM patronage
WHERE is_nsw_total = 1
GROUP BY mode_name
ORDER BY total_trips DESC;

-- Monthly trend for trains (state-wide)
SELECT year, month, SUM(trips) AS monthly_trips
FROM patronage
WHERE mode_name = 'Train' AND is_nsw_total = 1
GROUP BY year, month
ORDER BY year, month;

-- Weekday vs weekend average, per mode (state-wide)
SELECT mode_name, is_weekend, AVG(trips) AS avg_trips
FROM patronage
WHERE is_nsw_total = 1
GROUP BY mode_name, is_weekend;

-- CBD vs Parramatta recovery (per-region rows — exclude the aggregate)
SELECT ti_region, year, SUM(trips) AS yearly_trips
FROM patronage
WHERE ti_region IN ('Sydney CBD','Parramatta')   -- these are region rows, not All - NSW
GROUP BY ti_region, year
ORDER BY ti_region, year;

-- Bonus (needs the optional hourly table): the peak-hour profile,
-- and how it flattened post-COVID
SELECT tap_hour, AVG(CAST(Tap_Ons AS INTEGER)) AS avg_tap_ons
FROM patronage_hourly
WHERE mode_name = 'Train' AND ti_region = 'All - NSW'
GROUP BY tap_hour
ORDER BY tap_hour;
```

> The `is_nsw_total` filter is the one to remember: state-wide questions use `is_nsw_total = 1`; region-comparison questions use the named regions (which are automatically `is_nsw_total = 0`). Mixing them double-counts — a classic real-world data trap, and a great line for the README.

✅ **Checkpoint:** `opal.db` exists, SQL queries return sensible numbers. Commit: `"Clean data and load into SQLite; add exploration queries"`.

Module 4 — Explore & visualise (~2 hours)

Goal: find the story and produce charts for the README.

4.1 The headline story: COVID and recovery

In `notebooks/02_analysis.ipynb`:

```
import pandas as pd, sqlite3
import matplotlib.pyplot as plt

conn = sqlite3.connect("../data/opal.db")
monthly = pd.read_sql("""
    SELECT year, month, mode_name, SUM(trips) AS trips
    FROM patronage
    WHERE data_quality_flag = 0 AND is_nsw_total = 1
    GROUP BY year, month, mode_name
""", conn)
monthly['date'] = pd.to_datetime(dict(year=monthly.year, month=monthly.month, day=1))

fig, ax = plt.subplots(figsize=(11,6))
for mode, g in monthly.groupby('mode_name'):
    ax.plot(g['date'], g['trips'], label=mode)
ax.set_title("Sydney Opal patronage by mode, 2020 onward")
ax.set_xlabel("Date"); ax.set_ylabel("Monthly trips"); ax.legend()
plt.savefig("../images/patronage_by_mode.png", dpi=150, bbox_inches="tight")
plt.show()
```

4.2 Questions worth answering (pick 3–4, don't do all)

- How far did each mode fall in 2020, and which recovered fastest?
- Did the weekday/weekend split change after COVID? (working-from-home signal)
- How does CBD recovery compare to Parramatta or Chatswood?

- Are there seasonal patterns (summer dips, school terms)?

4.3 Save every chart you want in the README to `images/`

These PNGs are what make your GitHub README look like a finished report rather than raw code.

✅ **Checkpoint:** 3–4 clear charts saved, each answering a stated question. Commit: `"Add exploratory analysis and charts"`.

Module 5 — A small, honest predictive model (~2 hours)

Goal: one model you fully understand and can defend. *Not* a Kaggle leaderboard chase.

5.1 Frame the task

"Predict daily patronage for a given mode from the calendar features (day of week, month, year)." Simple, explainable, genuinely useful.

5.2 Build it

In `notebooks/03_model.ipynb`:

```
import pandas as pd, sqlite3
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, r2_score

conn = sqlite3.connect("../data/opal.db")
# One row per date: the state-wide daily Train series (aggregate rows only)
df = pd.read_sql("""
    SELECT * FROM patronage
    WHERE data_quality_flag = 0 AND is_nsw_total = 1
""", conn)
df['trip_origin_date'] = pd.to_datetime(df['trip_origin_date'])

train_df = df[df['mode_name'] == 'Train'].copy()
train_df['dow']   = train_df['trip_origin_date'].dt.dayofweek
train_df['month'] = train_df['trip_origin_date'].dt.month
train_df['year']  = train_df['trip_origin_date'].dt.year

X = train_df[['dow','month','year']]
y = train_df['trips']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
pred = model.predict(X_test)

print("MAE:", mean_absolute_error(y_test, pred))
print("R²:", r2_score(y_test, pred))
```

5.3 The most important part — interpret it honestly

Write a few sentences for the README: - What it predicts and how well (the actual MAE/R² numbers). - **What it can't do:** it only knows calendar patterns, so it won't anticipate lockdowns, strikes, fare changes, or major events. Say so plainly. - One honest next step: "adding weather or event data would likely improve it."

> Interviewers respect a candidate who states a model's limitations more than one who oversells it. This paragraph is worth more than the model itself.

5.4 Save the model for the app

```
import joblib
joblib.dump(model, "../app/patronage_model.pkl")
```

✅ **Checkpoint:** trained model + an honest write-up. Commit: `"Add patronage prediction model with evaluation"`.

Module 6 — The interactive website (~2–3 hours)

Goal: a live, public Streamlit app anyone can click around in. *This is the "visualise on a website" piece.*

6.1 Build the app

In `app/streamlit_app.py`:

```
import streamlit as st
import pandas as pd
import sqlite3
import plotly.express as px

st.set_page_config(page_title="Sydney Opal Patronage", layout="wide")
st.title("🚆 Sydney Opal Patronage Explorer")
st.caption("Data: Transport for NSW Open Data (CC BY). Built with SQL, Python, scikit-learn & Streamlit.")

@st.cache_data
def load_data():
    conn = sqlite3.connect("app/opal.db")  # copy opal.db into app/ for deploy
    df = pd.read_sql("SELECT * FROM patronage WHERE data_quality_flag = 0", conn)
    df['trip_origin_date'] = pd.to_datetime(df['trip_origin_date'])
    conn.close()
    return df

df = load_data()

# --- Filters ---
col1, col2 = st.columns(2)
with col1:
    mode = st.selectbox("Transport mode", sorted(df['mode_name'].unique()))
with col2:
    regions = sorted(df['ti_region'].unique())
    # Default to the state-wide series so visitors land on the full story.
    # (Filtering to ONE region at a time also avoids the All - NSW double-count trap.)
    default_ix = regions.index('All - NSW') if 'All - NSW' in regions else 0
    region = st.selectbox("Region", regions, index=default_ix)

date_range = st.slider(
    "Date range",
    min_value=df['trip_origin_date'].min().to_pydatetime(),
    max_value=df['trip_origin_date'].max().to_pydatetime(),
    value=(df['trip_origin_date'].min().to_pydatetime(),
           df['trip_origin_date'].max().to_pydatetime())
)

# --- Filter & plot ---
mask = (
    (df['mode_name'] == mode) &
    (df['ti_region'] == region) &
    (df['trip_origin_date'].between(date_range[0], date_range[1]))
)
filtered = df[mask].groupby('trip_origin_date', as_index=False)['trips'].sum()

fig = px.line(filtered, x='trip_origin_date', y='trips',
              title=f"{mode} patronage — {region}")
st.plotly_chart(fig, use_container_width=True)

# --- Quick stats ---
c1, c2, c3 = st.columns(3)
c1.metric("Total trips", f"{int(filtered['trips'].sum()):,}")
c2.metric("Daily average", f"{int(filtered['trips'].mean()):,}")
c3.metric("Peak day", f"{int(filtered['trips'].max()):,}")
```

6.2 Test locally

```
streamlit run app/streamlit_app.py
```

Your browser opens to a working interactive dashboard. Fix any column-name mismatches here.

6.3 Deploy free to the web

1. Copy `opal.db` into the `app/` folder so it ships with the code (keep it small).
2. Make sure `requirements.txt` is committed.
3. Push everything to GitHub.
4. Go to **share.streamlit.io**, sign in with GitHub, click **New app**, pick your repo, and set the path to `app/streamlit_app.py`.
5. Deploy. In a couple of minutes you get a public URL: `https://your-app.streamlit.app`.

> **Budget an hour for deploy friction.** Common snags: a package missing from `requirements.txt`, a wrong file path, or the data file too big. The free tier also "sleeps" after inactivity and wakes on the next visit — fine for a portfolio link.

✅ **Checkpoint:** a live URL you can send anyone. Commit: `"Add Streamlit app"`. **Put this URL in your CV and GitHub README.**

Module 7 — Optional Power BI parallel track (~1 hour)

Goal: keep "Power BI" credible on the CV with a real artifact.

- Open `data/opal.db` (or a cleaned CSV export) in Power BI Desktop.
- Build one clean page: a trips-over-time line chart with a mode slicer, a weekday/weekend bar chart, and a region breakdown.
- Export as PDF/PNG into `images/` and reference it in the README.
- If you have a Power BI service account, "Publish to web" for a live embed — otherwise the screenshot is enough.

Lead your live demo with Streamlit; use Power BI as the screenshot that backs up the CV keyword.

Module 8 — The README (~1 hour) — *don't skip this*

Goal: the document that turns code into a portfolio piece. For analyst roles, this is the single most-read file.

Structure

```
# Sydney Opal Patronage Explorer

🔗 **Live app:** https://your-app.streamlit.app
📊 Built with: SQL · Python (pandas) · scikit-learn · Plotly · Streamlit · Power BI

## The question
What happened to Sydney's public transport use through COVID and after —
and can we model daily patronage from calendar patterns?

## The data
Transport for NSW Opal Patronage (daily, 2020–present, CC BY).

## What I did
- Cleaned and combined N monthly files in Python
- Loaded into SQLite and aggregated with SQL
- Explored trends by mode, region, and weekday/weekend
- Built a random-forest model to predict daily patronage
- Deployed an interactive Streamlit dashboard

## What I found
[2–3 punchy findings + an embedded chart image]
![Patronage by mode](images/patronage_by_mode.png)

## Data quality note
Counts are rounded to the nearest 100, and small counts are privacy-suppressed
as "<50" — I treated these as 25 (the midpoint) and flagged affected rows.
Some dates under-report trips (missing-batch issues, per the dataset's
data-quality notes and confirmed by outlier checks); I flagged and excluded
these from the analysis.

## The model — and its limits
Predicts daily trips with MAE of X / R² of Y. It only knows calendar
features, so it can't anticipate lockdowns, strikes, or events. Adding
weather/event data is the obvious next step.

## Run it yourself
[setup + commands]
```

The README test

Could a recruiter understand the whole project in 60 seconds without running anything? If yes, it's done.

✅ **Checkpoint:** polished README with the live link and embedded charts. Commit: `"Add project README"`.

Module 9 — Make it interview-ready (~30 min)

Be able to answer, in two minutes each:

1. **"Walk me through this project."** → question → data → method → finding → limitation.
2. **"Why this dataset?"** → real, current, Sydney-relevant, a clear story (COVID recovery).
3. **"What was the hardest part?"** → (probably the download/cleaning or the deploy) — shows real hands-on work.
4. **"What would you do next?"** → weather/event features, finer geography, a proper time-series model.
5. **"What does your model *not* do?"** → the honesty answer that lands well.

Practising the walk-through out loud once or twice is worth more than another feature.

Final deliverables checklist

- [ ] Public GitHub repo with a sensible commit history
- [ ] Clean SQL queries in `sql/`
- [ ] Analysis notebooks with saved charts
- [ ] A trained, honestly-evaluated model
- [ ] A **live Streamlit URL**
- [ ] (Optional) a Power BI screenshot
- [ ] A README that tells the whole story in 60 seconds
- [ ] A two-minute spoken walk-through rehearsed

A few honest reminders

- **Finish before you embellish.** A complete small project beats an ambitious unfinished one — every time.
- **Don't oversell the AI.** A simple model you can fully explain is a strength; a complex one you can't defend is a trap.
- **The README and the live link are what get clicked.** Spend real time on them.
- **Commit often.** The commit history is itself evidence of how you work.
- **This is enough.** A clean repo + a live dashboard + a clear story is a genuinely strong analyst portfolio piece. He doesn't need five of these — he needs one, done well.

Good luck — this is very achievable, and it directly answers the "is your stack current?" question a three-year gap raises.